

강의 자료 · 쉬운 요약

하네스 엔지니어링, 진짜 쉽게

나만의 Claude Code 하네스 · 단계별 용어 풀이와 프롬프트

Red

Green

Refactor

목차

시작하기	하네스가 뭔데?	4
용어	어려운 용어 → 쉬운 말 번역표	6
한눈에 보기	전체 흐름 한 줄	9
1단계	AI에게 "우리 집 규칙" 알려주기 (CLAUDE.md)	10
2단계	자주 쓰는 일은 "레시피 카드"로 (Skill)	13
3단계	커밋할 때 자동 검문소 (husky + lint-staged + commitlint)	16
4단계	뭘 만들지 "AI가 나를 인터뷰" (spec)	21

5단계	디자인 규칙도 글로 적기 (design.md + Hook)	25
6단계	설계도 한 장 + "안 만들 것" 목록 (PRD + ADR + Out of Scope)	29
7단계	일을 "케이크 세로로" 자르기 (이슈 분해)	44
8단계	시험 문제 먼저, 공부는 나중 (TDD)	47
9단계	진짜 손님처럼 써보기 (E2E)	59
10단계	PR → 남의 컴퓨터 검사(CI) → 합치기	64
보너스	다 묶어서 한 방에 (16~17강)	71

이 자료 읽는 법

원본 PDF(17강, 226쪽)의 핵심만 추린 버전입니다. 각 단계는 같은 순서로 되어 있습니다.

1. 📖 **이게 뭐야?** — 단계에 나오는 용어를 하나씩 풀어서 설명
2. ? **왜 필요해?** — 안 하면 무슨 일이 생기는지
3. 🛠️ **할 일** — 실제로 하는 것
4. ⚠️ **자주 하는 실수**
5. 💬 **클로드에게 이렇게 말하기** — 복붙용 프롬프트

시작하기

하네스가 뭔데?

이게 뭐야?

하네스(harness) 는 원래 **말(馬)에 채우는 마구**, 즉 고삐·안장·굴레를 통틀어 부르는 말입니다. 말은 힘이 세고 빠르지만, 고삐가 없으면 **자기 가고 싶은 데로** 달립니다.

AI도 똑같습니다. 요즘 AI(모델)는 이미 충분히 똑똑합니다. 문제는 **엉뚱한 방향으로 열심히 달린다는** 것입니다. 그래서 업계에서는 AI 에이전트를 통제하는 모든 장치를 묶어 **하네스**라고 부르고, 이걸 설계하는 일을 **하네스 엔지니어링**이라고 합니다.

AI 에이전트 = 똑똑한 모델 + 하네스(고삐)

같은 모델을 써도 **하네스가 좋은 사람**이 훨씬 좋은 결과를 냅니다. 차이는 모델이 아니라 하네스에서 나옵니다.

하네스를 이루는 4가지

구성	쉬운 말	이 강의에서 만드는 것
가이드	"우리 집 규칙은 이거야" 미리 알려주기	CLAUDE.md, 스킴, 디자인 문서, PRD
검증	"규칙 어기면 못 지나가" 자동으로 막기	커밋 검사, 테스트, CI
피드백 루프	"틀렸으면 다시 해" 고치게 만들기	테스트 실패 → 수정 → 재실행
사람 개입점	"중요한 건 내가 정할게" 멈추고 묻기	승인 게이트, 기술 결정, PR 리뷰

참고: 학계에서 말하는 하네스는 메모리·플래닝·샌드박스·다중 에이전트까지 훨씬 넓습니다. 이 강의는 **실제로 바로 만들 수 있는 실용 영역만** 다룹니다.

어려운 용어 → 쉬운 말 번역표

원래 용어	쉬운 말	자세한 설명
CLAUDE.md	AI용 신입사원 안내서	1단계
Skill (스킬)	AI에게 주는 레시피 카드	2단계
Hook (훅)	자동 감시카메라	3단계, 5단계
husky / lint-staged	커밋 검문소	3단계
commitlint / Conventional Commits	커밋 메시지 맞춤법 검사	3단계
spec-original / spec-fixed	처음 주문서 / 확정 주문서	4단계
디자인 시스템	디자인 규칙집	5단계
PRD	설계도 한 장	6단계

원래 용어	쉬운 말	자세한 설명
ADR	"왜 이렇게 정했나" 메모	6단계
Out of Scope	"이번엔 안 만들 것" 목록	6단계
수직 슬라이싱	케이크 세로로 자르기	7단계
Acceptance Criteria (AC)	완료 체크리스트	7단계
Given-When-Then	"~일 때 / ~하면 / ~된다"	7단계
TDD	시험 문제 먼저 내고 공부하기	8단계
Red / Green / Refactor	빨간불 → 초록불 → 정리정돈	8단계
YAGNI	"안 시킨 건 하지 마"	8단계
Stub / Mock	가짜 부품 / 가짜 상대역	8단계
커버리지	시험 범위 중 실제로 풀어본 비율	8단계

원래 용어	쉬운 말	자세한 설명
Agent (에이전트)	검사관	8단계
E2E	진짜 손님처럼 처음부터 끝까지 써보기	9단계
PR	"제 작업 합쳐도 될까요?" 신청서	10단계
CI	남의 깨끗한 컴퓨터에서 다시 검사	10단계
브랜치 보호	검사 통과 못 하면 합치기 버튼 잠금	10단계

전체 흐름 한 줄

규칙 알려주기 → 자동 검문소 → 뭘 만들지 확정 → 디자인 규칙 → 설계도 → 작게 쪼개기
→ 시험 문제 먼저 → 통과시키기 → 정리 → 진짜 써보기 → 남의 컴퓨터 검사 → 합치기

구간	단계	몇 번 하나
환경 셋업	1~3단계	프로젝트 처음에 한 번
기획	4~7단계	기능 하나마다 한 번
구현	8단계	이슈 하나마다 반복
완성	9~10단계	기능이 다 끝나면 한 번

AI에게 "우리 집 규칙" 알려주기 (CLAUDE.md)

📖 이게 뭐야?

컨텍스트 윈도우 (Context Window) AI가 한 번에 머릿속에 담아둘 수 있는 글의 양입니다. 지금 대화 내용, 읽은 파일, 내가 한 말이 전부 여기 들어갑니다. 대화창을 닫거나 `/clear` 하면 **전부 사라집니다**. 버그가 아니라 원래 그렇게 설계된 것입니다. → 어제 "컴포넌트는 named export로 만들어" 라고 말했어도, 오늘 새 대화에서는 **모릅니다**. (금붕어 기억력)

CLAUDE.md 프로젝트 폴더 맨 위에 두는 **마크다운 파일**입니다. 클로드 코드는 **대화를 시작할 때마다 이 파일을 자동으로 읽습니다**. 그래서 여기 적어둔 규칙은 매번 말하지 않아도 지킵니다. 회사에 신입이 오면 주는 **온보딩 안내서**와 같습니다.

	README.md	CLAUDE.md
읽는 사람	사람	AI
내용	프로젝트 소개, 설치 방법, 라이선스	코딩 규칙, 폴더 구조, 명령어, 기술 결정
자동으로 읽히나	✗	✓ 매 세션 시작 시

권장 구성 4가지: ① 프로젝트 개요 ② 파일 구조 ③ 코딩 컨벤션 ④ 빌드·테스트 명령어. 200줄 이하로 유지합니다 (너무 길면 AI가 중요한 걸 놓침).

컨벤션 (Convention) 팀이 정한 코드 쓰는 약속입니다. 예: "컴포넌트는 named export", "이벤트 함수는 handle로 시작". 없으면 AI는 파일마다 `export default`, `export function`, `const` 를 그때그때 그럴듯하게 섞어 씁니다.

`/init` 클로드 코드의 기본 명령어입니다. `package.json`, `tsconfig.json` 같은 핵심 파일을 훑어서 **CLAUDE.md** 초안을 자동으로 써줍니다. 단, 초안입니다. 그대로 쓰면 바이브 코딩과 똑같은 실수입니다. 반드시 사람이 읽고 고칩니다.

| ? 왜 필요해?

안내서가 없으면 AI는 **매번 다른 스타일**로 코드를 씁니다. 이건 AI 잘못이 아니라 **지침이 없어서**입니다.

| 🛠 할 일

1. `/init` 으로 초안 만들기
2. 초안을 **사람이 읽고** 틀린 곳 고치기, 우리 팀 규칙 추가
3. 200줄 이하 유지. 새 규칙을 발견할 때마다 계속 추가 (**살아 있는 문서**)

! 자주 하는 실수

- `/init` 결과를 안 읽고 그대로 둔다
- 300~500줄짜리 백과사전으로 만든다

💬 클로드에게 이렇게 말하기

```
/init
```

방금 만든 CLAUDE.md를 보강해줘.

- 컴포넌트는 항상 named export로 만든다 (export default 금지)
- 이벤트 함수 이름은 handle로 시작한다 (handleClick)
- 테스트 실행: `npm test` / 개발 서버: `npm run dev`

200줄 넘지 않게 해줘.

자주 쓰는 일은 "레시피 카드"로 (Skill)

이게 뭐야?

Skill (스킬) 클로드에게 주는 **실행 지시서** = 레시피 카드입니다. "이 일을 할 때는 1번, 2번, 3번 순서로 해" 를 파일로 적어두고, **/이름** 한 마디로 꺼내 씁니다. 예: `/mermaid-diagram` 이라고 치면 → 클로드가 `.claude/skills/mermaid-diagram/SKILL.md` 를 읽고 → 적힌 순서대로 실행.

SKILL.md 스킬의 본문 파일입니다. 맨 위에 이름표(frontmatter)가 있고, 아래에 절차가 있습니다.

```
name: mermaid-diagram
description: src 구조를 분석해 아키텍처 다이어그램을 만든다
---
1. src/ 디렉토리 스캔 ...
2. docs/architecture/index.html 생성 ...
3. 브라우저에서 열기 ...
```

- **name** : 슬래시 명령 이름

- `description` : 언제 이 스킬을 쓰는지 설명. 클로드가 이걸 보고 "지금 이 스킬 쓸 때다" 하고 자동으로 꺼내기도 함

스킬 설치 위치

	프로젝트 스킬	글로벌 스킬
경로	<code>프로젝트/.claude/skills/이름/SKILL.md</code>	<code>~/.claude/skills/이름/SKILL.md</code>
쓸 수 있는 곳	이 프로젝트만	내 컴퓨터의 모든 프로젝트
공유	Git에 올리면 팀 전체 공유	나 혼자
예시	mermaid-diagram, tdd-red	ppt-maker

Mermaid (머메이드) 글자로 다이어그램을 그리는 문법입니다. `A --> B` 라고 쓰면 화살표 그림이 됩니다. 그림 파일(png)은 코드가 바뀌면 금방 낡지만, Mermaid는 텍스트라서 **Git으로 변경 추적**이 되고, **AI가 코드를 보고 바로 다시 그릴 수** 있습니다.

| ? 왜 필요해?

같은 부탁을 매번 길게 타이핑하면 **지치고, 빠뜨리고, 매번 조금씩 다르게** 시키게 됩니다. 레시피로 만들어 두면 **항상 같은 품질로** 실행됩니다.

! 자주 하는 실수

- 스킬을 한 줄짜리로 만든다 ("다이어그램 그려줘") → 그림 스킬 만든 의미가 없음. **순서를 구체적으로** 적어야 함

💬 클로드에게 이렇게 말하기

.claude/skills/mermaid-diagram/SKILL.md 스킬을 만들어줘.

하는 일:

1. src/ 의 컴포넌트, 훅, 유틸 파일 목록을 모으고 import 관계로 의존성 그래프 구성
2. docs/architecture/index.html 에 Mermaid flowchart TD 로 그리기 (CDN: unpkg.com/mermaid, dark 테마)
3. 브라우저로 열기 (mac: open / linux: xdg-open / windows: start)

그다음엔 그냥:

```
/mermaid-diagram
```

커밋할 때 자동 검문소 (husky + lint-staged + commitlint)

📖 이게 뭐야?

커밋 (Commit) Git에서 "지금 상태를 저장" 하는 것. 게임의 세이프 포인트와 같습니다. 커밋할 때 **메시지**(뭘 바꿨는지 한 줄 설명)를 같이 남깁니다.

Git Hook (깃 훅) Git이 특정 순간에 **자동으로 실행하는 스크립트**입니다. 스크립트가 **성공(exit 0)** 하면 계속 진행, **실패(exit 1)** 하면 커밋을 막습니다.

훅 이름	실행 시점	주로 하는 일
pre-commit	커밋이 만들어지기 직전	코드 검사
commit-msg	커밋 메시지가 기록되기 직전	메시지 형식 검사

```
git commit
├─ .git/hooks/pre-commit 실행
│  ├─ exit 0 → 커밋 진행
│  └─ exit 1 → 커밋 차단
```

husky (허스키) Git 혹은 원래 `.git/hooks/` 폴더에 있는데, 이 폴더는 **Git에 올라가지 않습니다**. 그래서 팀원이 코드를 받아도 훅이 없습니다. husky는 훅을 `.husky/` 폴더에 두고 **Git에 올릴 수 있게** 해주는 도구입니다. 팀원이 `npm install` 만 하면 **모두 같은 훅**이 깔립니다.

Lint (린트) / ESLint / Prettier

- **Lint:** 코드에서 **실수나 나쁜 습관을 찾아주는 검사** (안 쓰는 변수, 잘못된 문법 등). 맞춤법 검사기 같은 것
- **ESLint:** 자바스크립트/타입스크립트용 린트 도구. `--fix` 를 붙이면 고칠 수 있는 건 자동으로 고침
- **Prettier:** 코드 **모양 정리** 도구 (들여쓰기, 따옴표, 줄바꿈 통일)

lint-staged 프로젝트 전체를 검사하면 느립니다. lint-staged는 **이번에 커밋하려고 올린(staged) 파일만** 골라서 검사합니다. 빠르고 정확합니다.

```
pre-commit → lint-staged: staged된 .ts/.tsx 파일만
├─ ESLint --fix      (고칠 수 있는 건 고침)
├─ Prettier --write (모양 정리)
├─ 문제 없음        → 커밋 완료
└─ 자동 수정 불가   → 커밋 차단
```

Conventional Commits (컨벤셔널 커밋) 커밋 메시지를 쓰는 **국제 표준 형식**입니다. `타입(범위): 설명`

feat(tag): 노트에 태그 추가 기능 구현

fix(tag): 중복 태그 추가 시 에러 수정

타입

뜻

feat

새 기능

fix

버그 고침

test

테스트 추가/수정

refactor

동작은 그대로, 구조만 개선

chore

빌드, 설정, 패키지 변경

docs

문서

commitlint 커밋 메시지가 위 형식을 지켰는지 검사하는 도구. `commit-msg` 훅에서 실행되고, 틀리면 **커밋을 막습니다**.

? 왜 필요해?

AI가 만든 코드에 에러가 있어도 **아무도 알려주지 않습니다.** `npm run lint` 를 깜빡하면 그냥 커밋됩니다. **사람이 기억해야 하는 일은 시스템에 맡긴다** — 이게 하네스의 기본 철학입니다.

🔧 전체 파이프라인

```
git commit -m "feat(tag): 태그 추가"
[1] pre-commit → lint-staged  코드 검사
[2] commit-msg → commitlint   메시지 형식 검사
둘 다 통과 시 커밋 완료
```

그리고 CLAUDE.md에 커밋 규칙을 적어두면 **AI도 처음부터 형식을 맞춰** 씁니다.

클로드에게 이렇게 말하기

husky, lint-staged, commitlint 설치하고 설정해줘.

- 커밋 전: 바뀐 .ts/.tsx 파일만 eslint --fix, prettier --write
- 커밋 메시지: Conventional Commits 형식 강제
- CLAUDE.md에 커밋 메시지 규칙도 추가해줘

설정 끝나면 일부러 틀린 메시지로 커밋해서 막히는지 보여줘.

뭘 만들지 "AI가 나를 인터뷰" (spec)

📖 이게 뭐야?

요구사항 (Requirements) "무엇을 만들어야 하는가" 입니다. 코드는 "어떻게" 만드는가이고요. AI 시대에는 코드가 엄청 빨리 쌓이기 때문에 **방향 (무엇을)** 이 더 중요해졌습니다.

- **기능 요구사항:** 시스템이 **무엇을 해야** 하는가 (예: 태그를 추가할 수 있다)
- **비기능 요구사항:** 어떤 **조건을 만족해야** 하는가 (예: 0.1초 안에 표시된다, 태그는 최대 5개)

spec (스펙, specification = 명세서) 요구사항을 글로 적은 문서입니다. 이 강의에서는 두 개로 나눕니다.

파일	뜻	누가 쓰나
spec-original.md	처음 주문서 — 처음 받은 기능 설명 그대로	사람 (기획자/고객)
spec-fixed.md	확정 주문서 — 인터뷰 후 모든 결정이 확정된 명세	AI가 인터뷰 결과로 정리

AI 인터뷰 보통은 사람이 AI에게 질문합니다. 여기서는 **반대로 AI가 사람에게 질문**하게 합니다. AI는 수많은 프로젝트의 실패 패턴을 학습했기 때문에 사람이 놓친 **엣지 케이스**를 잘 짚어냅니다. (엣지 케이스: 빈 값, 너무 긴 값, 중복, 동시에 두 명이 수정 같은 **경계 상황**)

좋은 인터뷰 프롬프트 3가지 패턴

패턴	이유
"한 번에 하나씩 질문해"	질문 10개를 한꺼번에 받으면 대충 답하게 됨
"추천 답과 이유도 같이"	내가 판단할 근거가 생기고, 결정이 빨라짐
"코드베이스 먼저 탐색해"	기존 코드와 안 맞는 엉뚱한 질문을 막음

Ubiquitous Language (유비쿼터스 언어 = 용어 통일) "태그", "라벨", "키워드"를 섞어 쓰면 사람도 AI도 헷갈립니다. spec-fixed.md 마지막에 **용어 정의** 섹션을 두고, 이후 모든 문서·코드에서 **같은 단어**를 씁니다.

추적성 (Traceability) "왜 태그 최대 개수가 5개죠?" 라는 질문에 **답할 수 있는 상태**를 말합니다. spec-original과 spec-fixed를 따로 두고 결정 이유를 적어두면, 나중에 "이 제약 바뀌도 되나?" 를 판단할 수 있습니다.

| ? 왜 필요해?

"태그 기능 만들어줘" 한 마디면 AI가 **제멋대로** 정합니다. 별도 테이블? 배열? 최대 몇 개? 대소문자 구분? 잘못된 결정을 **언제 발견하느냐**에 따라 고치는 비용이 달라집니다.

발견 시점	고치는 비용
요구사항 단계	1배
설계 단계	5배
구현 단계	10배
테스트 단계	15배
배포 후	100배

→ 요구사항 단계의 1시간이 배포 후 100시간을 아깁니다.

클로드에게 이렇게 말하기

docs/features/tag/spec-original.md 를 읽고, 코드베이스를 먼저 살펴본 다음,

구현 전에 확인해야 할 것들을 나한테 질문해줘.

- 한 번에 하나씩만 질문해
- 각 질문마다 너의 추천 답과 이유도 같이 말해줘

인터뷰가 끝나면 결정한 내용(각 결정의 이유 포함)과 용어 정의를 spec-fixed.md 로 정리해줘.

디자인 규칙도 글로 적기 (design.md + Hook)

📖 이게 뭐야?

디자인 시스템 (Design System) 색, 글자 크기, 간격, 버튼 모양 같은 디자인 규칙을 한곳에 모아둔 규칙집입니다. 이 강의에서는 4개 파일로 나눕니다.

파일	내용	예시
colors.md	쓸 수 있는 색	주색 <code>indigo-600</code> , 배경 <code>gray-50</code>
typography.md	글자 크기·굵기	제목 <code>text-xl font-bold</code>
spacing.md	여백·간격	카드 안쪽 <code>p-4</code> , 요소 사이 <code>gap-2</code>
components.md	컴포넌트별 모양	태그 칩: 기본/hover/비활성/에러 상태별 클래스

design.md 기능 하나의 시각 명세서입니다 (`docs/features/{기능}/design.md`). 디자인 이미지를 클로드가 분석해서 **글로 바꾼 것**입니다. "회색 버튼"이 아니라 " `bg-gray-200` " 처럼 **정확한 클래스 이름**으로 적기 때문에 gray-200인지 gray-300인지 추측할 필요가 없습니다.

단일 진실의 원천 (Single Source of Truth) "이게 정답이다" 라고 할 수 있는 **딱 하나의 기준**. 기준이 이미지, 말, 기억에 흩어져 있으면 매번 다른 결과가 나옵니다.

Tailwind CSS `bg-indigo-600`, `p-4` 같은 **짧은 클래스 이름을 조합해서** 스타일을 입히는 CSS 도구. 클래스 이름이 곧 디자인 값이라 규칙으로 적기 좋습니다.

Claude Code Hook (클로드 코드 훅) 3단계의 Git Hook과 **같은 원리**인데, 이번엔 **클로드가 뭔가 할 때** 자동 실행됩니다. `.claude/settings.json` 에 등록합니다.

이벤트	실행 시점	주로 하는 일
UserPromptSubmit	내가 프롬프트를 보냈을 때	추가 정보 끼워 넣기
PreToolUse	클로드가 도구(파일 수정 등)를 쓰기 전	위험한 작업 차단
PostToolUse	도구를 쓴 직후	검사, 포맷 정리, 린트
Stop	클로드가 답변을 마칠 때	마무리 작업

matcher 어떤 도구에만 훅을 걸지 고르는 필터입니다. `"Edit!Write"` 면 **파일을 수정하거나 새로 만들 때만** 실행.

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{ "type": "command", "command": "node scripts/check-design-system.js" }]
    }]
  }
}
```

→ 클로드가 파일을 쓰는 순간 디자인 규칙 검사 → 위반하면 클로드가 그 결과를 보고 스스로 고칩니다.

| ? 왜 필요해?

테스트는 **동작**(버튼 누르면 태그가 추가되나)은 검사하지만 **외형**(칩 색이 맞나)은 검사하기 어렵습니다. 디자인 이미지 한 장만 주면 AI는 매번 색·간격을 **대충 비슷하게** 바꿉니다. 그래서 **글로 된 규칙 + 자동 감시**가 필요합니다.

흐름

디자인 이미지(Figma/Stitch/스크린샷) → design.md → 규칙 md 4종 → 스킴(따르게) → PostToolUse 훅(지켰나 검사)

클로드에게 이렇게 말하기

이 디자인 이미지를 보고 docs/design/ 아래에
colors.md, typography.md, spacing.md, components.md 로 디자인 규칙을 정리해줘.
Tailwind 클래스 이름으로 정확하게 적고, 상태별(기본/hover/비활성/에러)로 나눠줘.

클로드가 파일을 Edit/Write 할 때마다 scripts/check-design-system.js 가
자동 실행되도록 .claude/settings.json 에 PostToolUse hook 을 걸어줘.
colors.md 허용 목록에 없는 색 클래스를 쓰면 위반 목록을 출력하게 해줘.

설계도 한 장 + "안 만들 것" 목록 (PRD + ADR + Out of Scope)

이 단계가 기획에서 **가장 어렵고 가장 중요합니다**. 용어부터 하나씩 풀니다.

PRD가 뭐야?

PRD = Product Requirements Document (제품 요구사항 문서)

한 기능에 대해 "**궁금하면 여기 한 장만 보면 된다**" 는 문서입니다. 이 강의에서는 "**설계도 한 장**" 이라고 부릅니다.

4단계에서 만든 spec-fixed.md는 "**무엇을**" 만들지만 담고 있습니다. 여기에 "**어떤 구조 위에**" 만들지(기술 결정)를 더해서 **하나의 문서**로 합친 것이 PRD입니다.

spec-fixed.md (무엇을) + 기술 결정 (어떤 구조로) = prd.md (단일 기준 문서)

이후 모든 이슈 분해, 테스트, 구현이 이 문서를 출발점으로 삼습니다. AI가 헛갈리면 이 문서를 다시 읽게 하면 됩니다.

PRD의 역사 (왜 "가벼운 PRD"인가)

시대	PRD의 모습
1980~90년대 · 워터폴	수백 페이지짜리 계약서 . Microsoft·IBM이 코딩 전에 모든 걸 못 박으려 함
2000년대 · 애자일	"긴 문서는 변화를 못 따라간다" → PRD 무용론 vs 가벼운 PRD 로 갈림
현재	Notion·Confluence· 저장소 안의 md 파일 로, 짧지만 맥락 있는 살아 있는 문서

이 강의는 **가벼운 PRD** 입니다. 짧게 쓰되, **결정의 "왜"까지** 담습니다.

PRD에 들어가는 5가지

#	섹션	무엇을 쓰나	어디서 가져오나
1	개요	이 기능의 목적과 범위 (2~3줄)	spec-fixed.md
2	사용자 스토리	사용자 관점의 흐름 "나는 ~로서, ~하고 싶다, 왜냐하면 ~"	spec-fixed.md
3	기술 결정	어떤 구조 위에 올릴지 + 왜 (← 여기가 ADR)	새로 정해야 함

#	섹션	무엇을 쓰나	어디서 가져오나
4	Out of Scope	이번에 안 만들 것	사람이 정함
5	용어 정의	spec-fixed.md 의 용어를 그대로 이어받음	spec-fixed.md

1, 2, 5는 spec-fixed.md에서 가져오면 됩니다. 4는 "안 할 것"을 적으면 됩니다. **3번 기술 결정만은 지금 새로 정해야 합니다.** 그래서 가장 어렵습니다.

사용자 스토리 (User Story) 예시

- 나는 메모를 많이 쓰는 사용자로서, 메모에 태그를 달고 싶다. 나중에 주제별로 모아 보기 위해서.
- 나는 사용자로서, 태그를 클릭하면 그 태그가 달린 메모만 보고 싶다.

📖 왜 기술 결정이 제일 중요해?

기술 결정은 복리로 작용합니다.

바뀌는 것

영향

요구사항이 바뀌면

코드 **일부만** 고치면 됨

기술 결정이 바뀌면

코드 **전체**를 다시 짜야 함

예: 태그를 "메모 안 배열"로 저장하기로 했다가 나중에 "별도 테이블"로 바꾸면, 저장·불러오기·화면·테스트를 **전부** 다시 짜야 합니다. 앞 이슈의 기술 결정 하나가 뒤 이슈들의 난이도를 전부 바꿉니다.

 기술 결정은 AI에게 맡기면 안 된다

AI가 모르는 것

왜 문제인가

팀의 기술 수준

AI는 멋진 설계를 좋아함. 팀이 감당 못하면 오히려 짐

로드맵

6개월 뒤에 어떤 기능이 추가될지 AI는 모름

비즈니스 제약

배포 일정, 유지보수할 사람 수를 AI는 모름

→ AI는 후보를 비교해서 제안하고, 사람이 고릅니다.

📖 ADR이 뭐야?

ADR = Architecture Decision Record (아키텍처 결정 기록)

"왜 이렇게 정했나" 메모입니다. 코드를 보면 "어떻게" 만들었는지는 알 수 있지만, "왜" 그렇게 했는지는 알 수 없습니다. ADR은 그 "왜"를 남깁니다.

ADR의 탄생

시기	무슨 일
2010년 전후	문서는 산더미인데 "왜 이 기술 썼어요?" 물으면 아무도 답을 못 함
2011년	Michael Nygard 가 블로그 글 「Documenting Architecture Decisions」에서 가볍고 실용적인 형식 제안. 원래 형태: Status(상태) · Context(맥락) · Decision(결정) · Consequences(결과)
2010년대 후반	MADR (Markdown ADR) 이 여기에 " 검토한 대안(Considered Options) " 을 추가 → 지금 널리 쓰는 형태

핵심 철학: 설계 결과물이 아니라 **결정을 내린 맥락과 트레이드오프**를 남긴다. (트레이드오프: 하나를 얻으면 하나를 잃는 관계. 예: 단순함을 얻고 확장성을 잃음)

ADR의 4요소

요소	쉬운 말	무엇을 쓰나
Context (맥락)	"그때 상황이 어땠냐면"	이 결정이 필요했던 이유, 당시의 제약과 요구
Alternatives (대안)	"다른 후보도 있었는데"	함께 검토한 안들과 각각 왜 안 골랐는지
Decision (결정)	"그래서 이걸로 정함"	최종 선택
Consequences (결과)	"그래서 생기는 일"	장점 그리고 단점, 앞으로 달라지는 것

⚠ Consequences에 단점을 빼먹는 실수가 가장 흔합니다. 단점을 적어야 나중에 "그때 이 단점을 알고도 골랐구나" 를 알 수 있습니다.

ADR은 어디에 쓰나? → PRD 안에

별도 ADR 파일을 만들지 않고, PRD의 "기술 결정" 섹션을 ADR 4요소로 씁니다. 파일 하나 = 단일 진실의 원천. 결정이 많아져서 PRD가 너무 길어지면 그때 분리합니다 (YAGNI, 8단계 참고).

📖 3가지 안을 같은 기준으로 비교하기

좋은 Alternatives를 쓰려면 후보를 같은 기준으로 나란히 놓고 비교해야 합니다. 태그 기능 예시 (7가지 기준):

비교 기준	A안: 메모 안 배열	B안: 별도 테이블	C안: 배열 + 캐시
1. 데이터 구조	<code>note.tags = ['react']</code>	<code>tags</code> 테이블 + 연결 테이블	배열 + 태그 목록 캐시
2. API 변경 지점	메모 저장/수정 확장	태그 API 새로 만들	메모 저장/수정 확장
3. 상태 관리 변경 지점	메모 store 내부	태그 store 새로 만들	메모 store + 캐시
4. 즉시 저장 동작	단순	복잡	중간
5. 컴포넌트 구조	기존 재사용	새로 여러 개	기존 재사용
6. 기존 패턴과 일관성	높음	중간	높음
7. 테스트 쉬움	높음	중간	높음

용어 풀이:

- **API:** 프로그램끼리 데이터를 주고받는 창구 (예: "메모 저장해줘" 요청)
- **상태 관리 / store:** 화면에 보여줄 데이터를 담아두는 보관함
- **CRUD:** Create(만들기) · Read(읽기) · Update(수정) · Delete(삭제)

완성된 ADR 예시 (PRD의 "기술 결정" 섹션)

기술 결정: 태그 저장 방식

Context (맥락)

- 메모에 태그를 최대 5개까지 달 수 있어야 한다 (spec-fixed.md)
- 현재 메모는 localStorage 에 배열로 저장되고 있다
- 이번 버전에는 태그 이름 수정, 태그 색상 기능이 없다
- 팀은 2명, 2주 안에 배포해야 한다

Alternatives (검토한 대안)

- B안 별도 테이블: 태그 이름 일괄 수정에 유리하지만, 이번 범위엔 그 기능이 없고 새 store-API가 많이 필요해 거절
- C안 배열 + 캐시: 태그 목록 조회가 빠르지만, 메모가 수백 개 수준이라 캐시가 필요 없고 동기화 버그 위험만 늘어 거절

Decision (결정)

- A안: 메모 객체 안에 `tags: string[]` 배열로 저장한다

Consequences (결과)

- 장점: 기존 메모 저장 코드 재사용, 테스트가 쉬움, 2주 안에 가능

- 단점: 나중에 "태그 이름 일괄 변경" 기능이 생기면 모든 메모를 돌며 고쳐야 함
- 달라지는 것: Note 타입에 tags 필드 추가, 기존 메모는 tags: [] 로 마이그레이션 필요

Out of Scope가 뭐야?

Out of Scope = 범위 밖 = "이번엔 안 만들 것" 목록

기획에서 **가장 중요한** 부분입니다. "하지 않을 것"을 적는 게 "할 것"을 적는 것만큼 중요합니다.

왜? AI는 **친절해서** "이런 것도 있으면 좋겠죠?" 하고 **안 시킨 기능을 알아서 만듭니다**. "태그 색상 커스터마이징은 범위 밖" 이라고 적어두지 않으면 → 색상 선택기가 생겨 있습니다. 그 코드는 테스트도 없고, 아무도 검증 안 했고, 나중에 유지보수 부담만 됩니다.

Out of Scope는 **AI의 범위 초과 구현을 막는 울타리**입니다.

예시:

Out of Scope (이번 버전에서 하지 않음)

- 태그 색상 커스터마이징
- 태그 이름 일괄 수정
- 태그 자동완성
- 여러 태그 동시 필터 (AND/OR)
- 태그 개수 통계

세 문서는 뭐가 달라?

문서	질문	비유
spec-fixed.md	무엇을 만드나?	확정 주문서
PRD	무엇을 + 어떤 구조로 + 어디까지?	설계도 한 장
ADR (PRD 안의 기술 결정 섹션)	왜 이 구조로 정했나?	결정 메모
Out of Scope (PRD 안)	무엇을 안 만드나?	울타리

완성된 prd.md 뼈대

태그 기능 PRD

1. 개요

메모에 태그를 달고, 태그로 메모를 걸러 볼 수 있게 한다.

2. 사용자 스토리

- 나는 사용자로서, 메모에 태그를 달고 싶다. 주제별로 모아 보기 위해서.
- 나는 사용자로서, 태그를 클릭하면 그 태그가 달린 메모만 보고 싶다.

3. 기술 결정

(위의 ADR 예시 - Context / Alternatives / Decision / Consequences)

4. Out of Scope

- 태그 색상, 태그 이름 수정, 자동완성, 다중 필터

5. 용어 정의

- 태그: 메모에 붙이는 1~20자 짧은 단어. 대소문자 구분 없음

- 태그 칩: 화면에 태그를 보여주는 둥근 버튼 모양
- 필터: 선택한 태그가 달린 메모만 보여주는 것

할 일 (4단계)

1. **PRD 뼈대** — spec-fixed.md 기반으로 생성, 기술 결정은 TODO로 비워둠
2. **3안 비교 & 선택** — AI에게 7가지 기준으로 비교표 요청 → **사람이 고름**
3. **ADR 4요소 작성** — Context · Alternatives · Decision · Consequences (단점 필수)
4. **Out of Scope 확정 & 커밋** — docs(tag): PRD 작성

자주 하는 실수

- 기술 결정을 AI가 고르게 둔다
- 대안 비교 없이 결정만 적는다 → "왜?"에 답할 수 없음
- Consequences에 장점만 적는다
- Out of Scope를 비워둔다 → AI가 알아서 기능을 추가함

💬 클로드에게 이렇게 말하기

spec-fixed.md 로 docs/features/tag/prd.md 를 만들어줘.

섹션: 1.개요 / 2.사용자 스토리 / 3.기술 결정 / 4.Out of Scope / 5.용어 정의

개요·사용자 스토리·용어 정의는 spec-fixed.md 에서 가져오고,

기술 결정은 TODO 로 비워둬.

태그 데이터를 저장하는 방법 3가지 안을 만들어서 아래 7가지 기준으로 표로 비교해줘.

데이터 구조 / API 변경 지점 / 상태 관리 변경 지점 / 즉시 저장 동작 /

컴포넌트 구조 / 기존 패턴과 일관성 / 테스트 쉬움

결정은 내가 할 테니 추천만 하고 이유를 말해줘.

(사람이 하나 고른 뒤)

A안으로 결정. prd.md 의 기술 결정 섹션을 ADR 4요소로 써줘.

- Context: 이 결정이 필요한 맥락과 당시의 제약
- Alternatives: B안, C안과 각각 거절한 이유
- Decision: 선택한 안
- Consequences: 장점과 단점 모두, 앞으로 달라지는 것

prd.md 의 Out of Scope 에 "태그 색상 커스터마이징, 태그 이름 수정, 자동완성, 다중 필터" 를 넣어줘.
그리고 PRD 전체를 다시 읽고, 범위가 spec-fixed.md 보다 넓어진 곳이 있으면 알려줘.

일을 "케이크 세로로" 자르기 (이슈 분해)

📖 이게 뭐야?

이슈 (Issue) GitHub에 등록하는 **할 일 카드**입니다. 제목, 설명, 완료 조건을 적고 번호(#1, #2...)가 붙습니다. 이 강의의 규칙: **이슈 하나 = PR 하나 = TDD 사이클 한 바퀴**

수평 슬라이싱 vs 수직 슬라이싱 기능을 쪼개는 두 가지 방법입니다. 3층 케이크(화면 / 로직 / 저장)를 떠올려 보세요.

	수평 슬라이싱 × (가로로 자르기)	수직 슬라이싱 ✓ (세로로 자르기)
쪼개는 법	층별로: #1 API, #2 상태관리, #3 화면, #4 연결	기능별로: #1 태그 추가/제거, #2 태그 표시, #3 필터
이슈 1개 끝나면	아무것도 동작 안 함 (#4까지 다 해야 동작)	사용자가 바로 써볼 수 있음
테스트	"API 레이어 수정"은 사용자 관점 테스트 불가	완결된 시나리오 테스트 가능

판단 질문 딱 하나: "이 이슈만 끝내면 사용자에게 보여줄 동작이 있나?"

- Yes → 수직 슬라이스. TDD 사이클을 돌릴 수 있다

- No → 다른 이슈와 합치거나 다시 나눈다

Acceptance Criteria (AC, 인수 조건 = 완료 체크리스트) "이게 다 되면 이 이슈는 끝" 이라는 **완료 조건 목록**입니다. 세 가지 역할을 합니다.

1. 이슈의 **완료 조건**
2. PR의 **체크리스트**
3. 다음 단계(테스트 시나리오)의 **씨앗**

Given-When-Then (GWT) AC를 쓰는 형식. "**~일 때(Given) / ~하면(When) / ~된다(Then)**"

```
## Acceptance Criteria
```

- [] Given 노트 편집 화면에서, When 태그 입력 후 Enter를 누르면, Then 노트에 태그가 추가되고 화면에 표시된다
- [] Given 이미 존재하는 태그를 입력하고, When Enter를 누르면, Then 중복 추가되지 않는다

gh CLI 터미널에서 GitHub를 다루는 공식 명령어 도구입니다. 클로드가 직접 실행할 수 있어서 이슈 여러 개를 **한 번에 등록**할 수 있습니다.

```
gh issue create --title "태그 추가/제거" --body "..." # 이슈 생성
gh issue list # 이슈 목록
gh pr create --title "feat(tag): 태그 추가/제거" # PR 생성
```

칸반 보드 (Kanban) 할 일 카드를 **Todo** → **In Progress** → **Done** 칸으로 옮기며 관리하는 게시판. GitHub Projects에서 쓸 수 있습니다.

클로드에게 이렇게 말하기

prd.md 를 보고 기능을 이슈로 쪼개서 docs/features/tag/issues.md 에 써줘.

- 수직 슬라이싱: 각 이슈는 그것만 끝내도 사용자가 확인할 수 있는 동작이 있어야 해
- 각 이슈에 Given-When-Then 형식의 Acceptance Criteria 를 달아줘
- 이슈끼리 의존성(어느 게 먼저인지)도 적어줘

issues.md 의 이슈들을 gh CLI 로 GitHub Issues 에 등록해줘.

 4~7단계를 매번 하기 귀찮으면 → `/feature-planner` 스킴으로 묶기 (8강). 단, 3군데는 반드시 멈추고 사람에게 물어보게(승인 게이트) 합니다: ① 3안 중 고르기 ② Out of Scope 확정 ③ 쪼개진 이슈 검토. **AI는 제안자, 사람은 결정자**. 게이트 없이 끝까지 달리면 되돌릴 수 없는 결정이 자동으로 내려집니다.

시험 문제 먼저, 공부는 나중 (TDD)

이 단계는 처음 들으면 어렵습니다. **비유만 이해해도 실습은 충분히 따라갈 수 있습니다.** 코드 예시는 "아 이렇게 생겼구나" 하고 구경만 해도 됩니다.

테스트 코드가 뭐야?

한 줄로: "이렇게 하면 → 이렇게 돼야 한다" 를 **컴퓨터가 대신 확인**하게 적어둔 것.

비유: **자판기 검사** 새 자판기를 설치하면 이렇게 확인합니다.

"1000원 넣고 콜라 버튼 누르면 → 콜라가 나와야 한다"

사람이 매번 손으로 확인하면 지치고 빠먹습니다. 그래서 이 확인 과정을 **코드로 적어두고** 명령 한 줄(`npm test`)로 1초 만에 다시 확인합니다.

```
test('1000원 넣고 콜라 버튼 → 콜라 나옴', () => {  
  vendingMachine.insert(1000);           // 1000원 넣고  
  const drink = vendingMachine.press('cola'); // 콜라 버튼 누르면  
  expect(drink).toBe('cola');             // 콜라가 나와야 한다  
});
```

- 맞으면 → 초록불 (통과)
- 틀리면 → 빨간불 (실패)

📖 TDD가 뭐야?

한 줄로: 시험 문제를 먼저 내고, 공부(코드)는 나중에 하는 방법. (TDD = Test-Driven Development, 테스트가 이끄는 개발)

왜 순서가 중요해? AI에게 "태그 기능 만들어줘" 하면 AI는 코드와 테스트를 동시에 만듭니다. 이건 학생이 시험 문제와 답안을 같이 만드는 것과 같습니다. 당연히 100점이 나오죠. 답이 틀려도 문제를 거기 맞춰서 냈으니까요.

그래서 시험 문제(테스트)를 먼저 확정하고, 코드는 그 시험에 맞춰서 짜게 만듭니다.

📖 Red → Green → Refactor (신호등 3단계)

단계	신호등	비유	하는 일
Red	●	시험지 먼저 받기	테스트를 먼저 쓴다. 아직 코드가 없으니 다 틀림
Green	●	시험 통과할 만큼만 공부	테스트가 통과할 딱 그만큼만 코드를 짤다
Refactor	🖌️	공책 정리	답은 그대로 두고 지저분한 코드만 깔끔하게

📖 Red: "다 틀린" 상태가 왜 좋은 거야?

처음에 테스트가 **다 틀려야** 합니다. 이상하게 들리지만 이유가 있습니다. 처음부터 통과하는 테스트는 **아무것도 검사하지 않는 엉터리 시험**일 수 있거든요. "틀렸다가 → 코드를 짜니까 → 맞았다"를 봐야 그 테스트를 믿을 수 있습니다.

그런데 "틀리는 방식"도 중요합니다

상태	비유	화면에 뜨는 말	좋은가?
올바른 Red	시험지는 멀쩡, 문제를 다 틀림	<code>not implemented</code> (아직 안 만들었어요)	✅ 정상 출발점

고장 (Broken)

시험지가 찢어져서 채점 자체를 못 함

Cannot find module (파일을 못 찾겠어요)

 고쳐야 함

Stub(스텝)이 뭐야? → "빈 껍데기 함수"

시험 문제는 "addTag 함수를 쓰면 태그가 추가돼야 한다" 인데, **addTag 함수가 아직 아예 없으면** 시험지가 찢어진 상태(Broken)가 됩니다.

그래서 **이름만 있고 속은 비어 있는 껍데기**를 먼저 만들어 둡니다.

비유: 콘센트만 달아놓고 전기는 아직 안 연결한 벽. 플러그는 꽂히지만 전기는 안 들어옴.

```
export function addTag(noteId: string, tagName: string): Promise<Note> {  
  throw new Error('not implemented'); // "아직 안 만들었어요"  
}
```

이렇게 하면 시험지가 멀쩡해져서 **"12문제 중 12문제 틀림"** 이 깔끔하게 나옵니다. 여기가 출발점입니다.

Green: "딱 그만큼만" 이 뭐야? → YAGNI

YAGNI = "You Aren't Gonna Need It" = "그거 안 쓸걸?"

비유: 이삿짐 싸기 "혹시 나중에 쓸지도 몰라" 하고 다 챙기면 짐은 산더미인데 1년 동안 한 번도 안 씁니다. 옮기고 정리하는 수고만 늘죠.

AI가 딱 이렇습니다. 태그 추가만 시켰는데 "색상도 넣어드렸어요! 자동완성도요!" 하고 덧붙입니다. 그 기능은 **아무도 시키지 않았고, 테스트도 없어서 맞는지 아무도 모릅니다.**

그래서 규칙은 하나입니다.

시험 문제(테스트)에 없는 건 만들지 않는다.

한 번에 **하나씩** 테스트 12개를 한꺼번에 통과시키지 않고 **하나씩** 통과시킵니다. 하나 고칠 때마다 전체 시험을 다시 봐서 **아까 맞던 문제가 틀려지지 않았는지** 확인합니다. 틀린 개수가 곧 진행 상황입니다: 12 → 11 → 10 → ... → 0 🎉

@ac-verifier가 뭐야? → "검사관"

시험을 다 통과했다고 진짜 끝난 걸까요? **시험 문제 자체가 뭔가를 빼먹었을 수도** 있습니다. 그래서 **검사관(에이전트)** 이 7단계에서 만든 **완료 체크리스트**를 하나씩 보면서 "진짜 다 됐나?" 를 따로 확인합니다. 빠진 게 있으면? → 다시 Red로 가서 시험 문제를 추가합니다.

스킬이랑 에이전트는 뭐가 달라?

	스킬	에이전트
비유	레시피 카드 (순서대로 따라 함)	검사관 (스스로 판단함)
부르는 법	<code>/tdd-red</code>	<code>@ac-verifier</code>

순서가 정해진 일은 스킬, 매번 판단이 필요한 일은 에이전트입니다.

 Refactor: 공책 정리

한 줄로: 답(동작)은 그대로 두고, 코드만 깔끔하게 정리하는 것. 급하게 통과시킨 코드는 지저분합니다. 같은 코드가 두 번 있거나, 이름이 `data`, `temp` 같이 대충 지어져 있죠.

규칙은 간단합니다.

하나 고치고 → 시험 다시 보기 → 틀리면 바로 되돌리기

시험(테스트)이 있으니까 마음 놓고 정리할 수 있습니다. 이게 테스트를 먼저 만든 덕분입니다.

📖 보안 점검

마지막으로 두 가지를 검사합니다.

- **타입 검사** (`npx tsc --noEmit`): "숫자 자리에 글자 넣은 곳 없나?" 같은 실수를 찾음
- **보안 검사** (`npm audit`): 내가 설치한 남의 코드(패키지)에 **알려진 보안 구멍**이 있나 확인

`/security-review` 한 줄로 실행하고, 결과는 **"지금 고쳐야 함 / 고치면 좋음 / 무시해도 됨"** 3개로 나뉘서 보입니다.

💬 클로드에게 이렇게 말하기

① 시험 범위 정하기

이슈 #1 을 보고 함수 시그니처(입력/출력/에러 메시지)를 먼저 제안해줘. 구현은 하지 마.
그다음 테스트 시나리오를 정상 / 경계 / 예외로 나눠서 목록으로 보여줘.
경계값은 최대-1, 최대, 최대+1 을 꼭 포함해.

② Red: 시험 문제 먼저

승인한 시나리오대로 Vitest 테스트를 작성해줘.

구현 코드는 절대 작성하지 말고, 함수는 `throw new Error('not implemented')` 로만 만들어줘.

`npm test` 돌려서 "N개 실패, 0개 통과" 인지 보여줘.

③ Green: 딱 통과할 만큼만

실패하는 테스트를 하나씩만 통과시켜줘.

테스트에 없는 기능은 절대 추가하지 마. 테스트 파일은 수정 금지.

하나 통과시킬 때마다 `npm test` 전체를 돌려서 다른 게 안 깨졌는지 확인해.

④ 검사관

@ac-verifier 이슈 #1 의 완료 체크리스트를 현재 코드와 테스트가 전부 만족하는지

독립적으로 검사해줘. 빠진 게 있으면 목록으로 알려줘.

⑤ Refactor: 공책 정리

동작은 바꾸지 말고 코드만 정리해줘.

기준: 중복 / 이름 / 너무 많은 일을 하는 함수 / 매직넘버 / CLAUDE.md 규칙.

한 번에 하나씩 고치고 매번 npm test. 깨지면 바로 되돌려.

⑥ 보안 점검

```
/security-review
```

이슈 하나 끝나면? 다음 이슈에 똑같은 순서를 반복합니다. 처음엔 30분, 두 번째부터는 5~10분이면 됩니다.

더 알고 싶은 사람만 (몰라도 실습 가능)

시그니처: 함수의 겉모양(이름, 입력, 출력, 에러). 구현 전에 먼저 정해두면 AI가 마음대로 바꾸지 못합니다.

```
addTag(noteId: string, tagName: string): Promise<Note>
// 태그 5개 초과 → '최대 태그 수를 초과했습니다'
// 중복 태그 → '이미 추가된 태그입니다'
// 빈 문자열 → '태그 이름을 입력해주세요'
```

경계값: 버그는 경계에서 납니다. 태그 최대 5개면 **4개, 5개, 6개**일 때를 모두 시험합니다. < 와 <= 한 글자 차이 버그는 딱 경계에서만 보이거든요.

```
[정상] addTag should add tag to note when valid tag is provided
[경계] addTag should add tag when note has exactly 4 tags (max-1)
[경계] addTag should accept tag with exactly 20 characters (max length)
[예외] addTag should throw '이미 추가된 태그입니다' when tag already exists
[예외] addTag should throw when note already has 5 tags
```

Stub vs Mock (테스트용 대역 배우): 테스트할 때 진짜 결제나 진짜 이메일을 보내면 곤란하니 가짜를 씁니다.

	비유	언제
Stub	정해진 대사만 하는 대역 ("이 값 줄게")	값을 받아올 때 (서버 응답, 현재 시간)

비유

언제

Mock

감시 카메라 달린 대역 ("너 나 몇 번 불렀어?")

뭔가 시켰는지 확인할 때 (결제, 이메일)

가짜를 너무 많이 쓰면 테스트는 통과하는데 실제 앱은 안 돌 수 있습니다.

Vitest / React Testing Library (RTL)

- Vitest: 테스트를 돌려서 채점하는 **채점기** (`npm test`)
- RTL: 화면을 **손님 눈으로** 검사하는 도구
 -  주방 안 검사: `expect(component.state.tags).toHaveLength(1)` (내부를 들여다봄)
 -  손님 자리 검사: `expect(screen.getByText('react')).toBeInTheDocument()` (화면에 보이냐?)
 - 찾는 순서: `getByRole` (버튼 같은 역할로) → `getByText` (보이는 글자로) → ... → `getById` (최후 수단)

커버리지: 테스트가 코드 중 몇 %를 실제로 지나갔는지. 점수 채우기용이 아니라 **"빠뜨린 테스트 없나?"** 힌트로 봅니다.

지표

뜻

Function

함수가 한 번이라도 불렀나 (제일 느슨)

Statement / Line

각 줄이 실행됐나

Branch

if/else 양쪽 길을 다 가봤나 (제일 엄격)

리팩토링 점검 기준 5가지: ① 중복 ② 이름(data, temp x) ③ 한 함수가 너무 많은 일 ④ 매직넘버:깊은 중첩 ⑤ CLAUDE.md 규칙 위반 (Martin Fowler의 정의: "외부 동작은 그대로, 내부 구조만 개선")

보안 결과 3등급: 즉시 수정(빌드 막는 타입 오류, 심각한 취약점, 비밀키 노출) / 권장 수정 / 무시 가능. `npm audit fix --force` 는 쓰지 마세요. 다른 패키지가 망가질 수 있습니다.

역사: TDD는 1990년대 Kent Beck이 정리했고, YAGNI도 그가 만든 XP(익스트림 프로그래밍)의 원칙입니다.

진짜 손님처럼 써보기 (E2E)

E2E가 뭐야?

한 줄로: 로봇이 진짜 브라우저를 열고, 진짜 손님처럼 클릭하고 타이핑해보는 테스트. (E2E = End-to-End, 처음부터 끝까지)

비유: 자동차

- 8단계 테스트: "엔진 정상? 바퀴 정상? 핸들 정상?" (부품 검사)
- E2E: "차에 타서 시동 걸고 실제로 달려지나?" (시운전)

부품이 다 멀쩡해도 조립이 잘못되면 차는 안 굴러갑니다. E2E가 그걸 잡아냅니다.

그럼 전부 E2E로 하면 되잖아?

안 됩니다. 시운전은 느리고 비싸기 때문입니다.

부품 검사 (단위 테스트)		시운전 (E2E)
속도	눈 깜짝할 새	몇 초~몇 분
실패하면	"바퀴 나사가 문제" 바로 앓	"어딘가 고장남" 만 앓
개수	많이	꼭 필요한 것만 조금

그래서 부품 검사는 많이, 시운전은 대표 코스 몇 개만 합니다.

시운전 코스는 어떻게 짜?

4가지만 기억하세요.

1. **처음부터 끝까지** 한 번에 (앱 열기 → 태그 입력 → Enter → 태그가 보임)
2. **정상적인 길만** (이상한 입력은 부품 검사가 담당)
3. **부품 검사와 겹치지 않게** ("화면에 태그가 보이냐" 만 확인)
4. **구체적으로** ("태그 기능 잘 됨" × → "react 치고 Enter → react 칩이 보임" ✓)

시운전에서 고장을 발견하면?

시운전 결과를 조작하지 말고, 고장 난 부품을 찾아서 고친다.

1. 어느 부품이 문제인지 찾는다
2. 그 부품의 **부품 검사(단위 테스트)**를 하나 추가한다 (Red)
3. 부품을 고친다 (Green)
4. 시운전을 다시 한다

E2E는 **발견**, 단위 테스트는 **재발 방지**입니다.

Playwright

이 강의에서 쓰는 **시운전 로봇** 이름입니다.

```
npx playwright test          # 화면 없이 빠르게 (평소)
npx playwright test --headed  # 로봇이 클릭하는 걸 눈으로 구경 (시연용)
```

💬 클로드에게 이렇게 말하기

Playwright 를 설치하고 prd.md 의 사용자 스토리대로 tests/tag.spec.ts 를 만들어줘.

- 정상 흐름만, 처음부터 끝까지 한 사용자 여정으로
 - 셀렉터는 `getByRole` > `getByText` > `getByPlaceholder` > `getByTestId` 순서
 - `afterEach` 에서 테스트가 만든 데이터를 지워줘
- 그리고 `npm run test:e2e` 로 돌려줘.

📦 더 알고 싶은 사람만 (몰라도 실습 가능)

테스트 피라미드 (Mike Cohn, 2009): 아래가 넓고 위가 좁은 삼각형.

```
/  E2E  \      ← 적게 · 느림 · 비쌈
/  통합 테스트  \    ← 부품끼리 연결 확인
/  단위 테스트   \  ← 많이 · 빠름 · 싼
```

태그 예시로 보면: 단위 = `addTag()` 가 맞는 값을 돌려주나 / 통합 = 저장 기능과 화면 상태가 연결되나 / E2E = "react" 치고 Enter → 칩이 보이나

Flaky 테스트: 코드는 그대로인데 **됐다 안 됐다** 하는 테스트. 인터넷 속도나 타이밍 때문에 생기고, 시운전에서 자주 생깁니다.

셀렉터 = 화면에서 버튼·글자를 찾는 방법

순위	방법	뜻
1	<code>getByRole</code>	"버튼"처럼 역할 로 찾기 (시각장애인용 화면 읽기 프로그램과 같은 방식)
2	<code>getByText</code>	보이는 글자 로 찾기
3	<code>getByPlaceholder</code>	입력창 안의 안내 문구 로 찾기
4	<code>getByTestId</code>	테스트 전용 이름표 (최후 수단)

데이터 격리: 테스트가 만든 메모는 끝나면 지웁니다(`afterEach`). 그래야 어떤 순서로 돌려도 결과가 같습니다.

Trace Viewer: 시운전이 실패하면 그 순간의 스크린샷·네트워크·콘솔 기록을 보여주는 블랙박스.

언제 만드나: 기능의 모든 이슈가 끝난 뒤 한 번 만들고, 이후엔 PR 올리기 전마다 다시 돌립니다.

PR → 남의 컴퓨터 검사(CI) → 합치기

브랜치가 뭐야?

한 줄로: 원본을 건드리지 않고 복사본에서 작업하는 것.

비유: 연습장 원본 공책(main)에 바로 쓰다 틀리면 지저분해집니다. 그래서 **연습장(브랜치)**에 먼저 풀어보고, 맞으면 원본에 옮겨 적습니다.

PR이 뭐야?

한 줄로: "연습장에 푼 거, 원본에 옮겨 적어도 될까요?" 신청서. (PR = Pull Request)

1. 연습장(브랜치)에서 작업한다
2. PR을 올린다
3. 동료가 보고(리뷰), 자동 검사(CI)가 돈다
4. 문제없으면 원본에 **합친다 (머지)**

PR 설명에 `Closes #1` 이라고 쓰면, 합쳐질 때 **1번 이슈가 자동으로 완료** 처리됩니다.

📖 CI가 뭐야?

한 줄로: PR을 올리면 **GitHub의 깨끗한 새 컴퓨터**가 처음부터 다시 설치하고 검사해 주는 것. (CI = Continuous Integration, 지속적 통합)

비유: 요리 대회 심사 "우리 집 부엌에선 맛있었는데요?" 는 증거가 안 됩니다. 우리 집에는 이미 양념이 다 있으니까요. **심사위원의 빈 부엌**에서 레시피대로만 다시 만들어도 맛있어야 진짜입니다.

CI는 빈 부엌에서 순서대로 검사합니다.

설치 → 코드 맞춤법 검사 → 타입 검사 → 보안 검사 → 테스트

전부 통과하면  초록불, 하나라도 실패하면  빨간불.

📖 브랜치 보호가 뭐야?

한 줄로: 빨간불이면 합치기 버튼 자체가 잠기는 GitHub 설정.

CI가 빨간불을 띄워도 누가 그냥 합치기 버튼을 누르면 소용없습니다. 그래서 **버튼을 잠가버립니다.**

- 원본(main)에 **직접 올리기 금지** → 반드시 PR을 거쳐야 함
- **CI 통과 전엔 합치기 버튼 비활성화**

사람의 의지력에 맡기지 않고 **시스템이 강제**합니다. 이게 하네스의 핵심입니다.

클로드에게 이렇게 말하기

CI 만들기

`.github/workflows/ci.yml` 을 만들어줘.

main 으로 가는 PR 마다: `npm ci → lint → tsc --noEmit → npm audit --audit-level=moderate → npm test`

PR 올리기 스킵 만들기

`/create-pr` 스킵을 만들어줘.

순서: E2E 실행 → 실패하면 중단하고 원인 안내 → 통과하면 `push → gh pr create`.

E2E 테스트 코드를 고쳐서 통과시키는 건 금지.

PR 본문에 "Closes #이슈번호" 넣고, 사용자가 "PR 올려줘"라고 해도 이 스킵이 실행되게 해줘.

브랜치 보호 는 클로드가 아니라 사람이 직접 설정합니다. GitHub 저장소 → **Settings** → **Branches** → **Add rule** → main 선택 → "Require a pull request before merging"  / "Require status checks to pass" 

더 알고 싶은 사람만 (몰라도 실습 가능)

브랜치 구조 예시

```
main
├── feature/tag-spec    ← 태그 기능 전체를 모으는 연습장
│   ├── feat/tag-1     ← 이슈 #1 연습장
│   └── feat/tag-2     ← 이슈 #2 연습장
```

ci.yml 전체 모습 (GitHub Actions = GitHub에 들어 있는 CI 도구)

```
name: CI
on:
  pull_request: { branches: [main] }
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      - run: npm ci
      - run: npm run lint
      - run: npx tsc --noEmit
      - run: npm audit --audit-level=moderate
      - run: npm test
```

검사

`npm run lint`

잡는 것

코드 맞춤법·스타일

`npx tsc --noEmit`

타입 실수

검사

잡는 것

`npm audit`

남의 코드(패키지)의 보안 구멍

`npm test`

테스트

npm ci vs npm install: `npm ci` 는 기록된 **정확한 버전 그대로** 설치합니다. 그래서 언제 돌려도 같은 결과가 나옵니다.

머지 방식: `gh pr merge --merge` (squash 아님). 이슈별 작업 기록을 그대로 남깁니다.

/create-pr 이 E2E 실패 시 안내할 순서: 블랙박스(Trace Viewer) 보기 → 어느 부품인지 찾기 → 단위 테스트 추가 → 고치기

지금까지 쌓은 자동 검사들

언제

무엇을

커밋할 때 (husky + lint-staged)

코드 스타일

커밋할 때 (commitlint)

커밋 메시지 형식

구현 끝날 때 (/security-review)

보안

언제

무엇을

PR 올리기 전 (/create-pr)

E2E 시운전

PR 올린 후 (CI)

빈 부역에서 전부 다시

문서 정리: 기능이 원본에 합쳐지면 "6개월 뒤에도 볼까?" 기준으로 정리합니다. README·운영 가이드·다이어그램은 남기고, 기능별 spec·PRD·이슈 작업표는 정리합니다 (이미 코드와 테스트에 다 들어 있으니).

다 묶어서 한 방에 (16~17강)

📖 이게 뭐야?

8단계에서 이슈 하나 끝내려면 명령을 **7번** 쳐야 했습니다.

```
/test-scenarios → /tdd-red → /tdd-green → @ac-verifier → /tdd-refactor → /security-review → /create-pr
```

이걸 **한 번에 묶은 스킬**이 `/tdd-loop` 입니다. 레시피 카드들을 모아 놓은 **레시피 북**이라고 보면 됩니다.

📖 두 가지 버전

	<code>/tdd-loop</code>	<code>/tdd-auto-loop</code>
비유	자동 순서표	자율주행
하는 일	순서대로 진행, 중간중간 "이거 괜찮아요?" 물어봄	안 묻고 끝까지 감

사람이 할 일

단계마다 OK 누르기

마지막에 PR 한 번 보기



자율주행은 언제 멈춰?

확실히 잘못됐을 때만 멈춥니다(STOP). 예를 들면:

- 시험 문제가 너무 적을 때 (5개 미만)
- 3번 시도해도 통과를 못 할 때
- 검사관이 "체크리스트 다 못 채웠어요" 라고 할 때
- 심각한 보안 문제가 있을 때

멈출 때는 **"여기서 이런 이유로 멈췄어요"** 를 남기고 깔끔하게 끝냅니다. 사람이 나중에 보고 고쳐서 다시 돌리면 됩니다.

잘 멈추는 게 자동화의 핵심입니다. 잘 쓴 이슈는 멈추지 않고 끝까지 갑니다.

💬 클로드에게 이렇게 말하기

/tdd-loop 스킬을 만들어줘. 이슈 번호를 받아서
브랜치 생성 → /test-scenarios → /tdd-red → /tdd-green → @ac-verifier
→ /tdd-refactor → /security-review → /create-pr 순서로 실행해.
각 스킬의 승인 질문은 건너뛰지 마. 어느 단계에서 멈췄는지 명확히 보고해.

📦 더 알고 싶은 사람만 (몰라도 실습 가능)

컨테이너 스킬: 다른 스킬들을 순서대로 부르지만 하는 스킬. 8강 `/feature-planner` 가 기획 4단계를 묶은 것과 같은 방식입니다.

서브에이전트: 메인 클로드가 따로 띄우는 **보조 클로드**. 자기 일만 하고 결과만 돌려준 뒤 사라집니다. 그래서 한 단계의 긴 코드나 에러 기록이 다음 단계로 섞여 들어가지 않습니다. `/tdd-auto-loop` 는 단계마다 보조 클로드를 띄워서 일을 시킵니다.

전체 STOP 조건: 시나리오 5개 미만 / Red가 파일 못 찾음 에러로 실패 / Green 3번 실패 / AC 검증 실패 / 리팩토링 후 테스트 실패 / 보안 재검사 후 심각 등급 이상 / 커밋 메시지 형식 실패

STOP vs 묻기

- × 묻기: "진행할까요?" 하고 기다림 → 사람이 바로 답하지 않으면 계속 멈춰 있음

- ✓ STOP: 이유를 남기고 깔끔히 끝냄 → 사람이 나중에 보고 고쳐서 다시 실행